FHIR & RESTful APIs for Clinical Information Systems

Workshop UPNA 2026 – Day 1

Prof. Dr. Rafael Mayoral Malmström

Hochschule Kempten – University of Applied Sciences

Spring 2026

Hochschule Kempten
University of Applied Sciences
Fakultät Informatik

upna
Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Erasmus+ Teaching Mobility 2026

Agenda – Day 1

1. Welcome, goals, and setup check
2. FHIR fundamentals
3. RESTful APIs in healthcare
4. Lab 1: Basic interaction with a FHIR server
5. Lab 2: Mini clinical workflow
6. Short code demo & wrap-up

Context: Clinical Information Systems

- Increasing digitalisation of healthcare
- Heterogeneous systems: HIS, LIS, PACS, medical devices, apps
- Need to exchange data across:
  - departments
  - institutions
  - even national borders
- Interoperability is a core challenge

Why Standards?

- Avoid bespoke, one-off interfaces
- Enable reuse of tools and components
- Reduce cost and complexity of integration
- Support patient safety (fewer mapping errors)
- Basis for:
  - clinical decision support
  - secondary use (research, quality management)
  - AI and analytics

What is FHIR?

FHIR = Fast Healthcare Interoperability Resources

- Modern standard by HL7
- Resource-based, composable building blocks
- Designed for web technologies (HTTP, REST, JSON/XML)
- Supports many use cases:
  - Patient administration
  - Orders & results
  - Observations & vital signs
  - Care plans, medication, etc.

Reference: `https://www.hl7.org/fhir/`

Resources: The Basic Building Blocks

- A **Resource** = small, well-defined data structure
- Examples:
  - `Patient`
  - `Observation`
  - `Condition`
  - `DiagnosticReport`
- Each resource has:
  - `resourceType`
  - `id`
  - metadata (`meta`, `text`, ...)
  - domain-specific data

Example: Patient (JSON)

```
{
        "resourceType": "Patient",
        "id": "12345",
        "name": [{
                "family": "Doe",
                "given": ["John"]
        }],
        "gender": "male",
        "birthDate": "1990-01-01"
}
```

- Minimal example
- Real-world patients have more attributes (identifiers, address, ...)
- `https://www.hl7.org/fhir/patient.html`

Example: Observation (JSON)

```
{
        "resourceType": "Observation",
        "status": "final",
        "code": {
                "coding": [{
                        "system": "http://loinc.org",
                        "code": "85354-9",
                        "display": "Blood pressure panel"
                }]
        },
        "subject": { "reference": "Patient/12345" },
        "effectiveDateTime": "2024-05-10T10:00:00Z",
        "valueQuantity": {
                "value": 120,
                "unit": "mmHg"
        }
}
```

- Reference patient via `subject.reference`; Use LOINC for coding the measurement type
- https://www.hl7.org/fhir/observation.html

References and Linking

- Resources can reference each other:
  - `Observation.subject` → `Patient`
  - `DiagnosticReport.result` → `Observation`
- Enables modeling of clinical workflows:
  - Admission, diagnostic tests, reports, follow-up

REST in a Nutshell

- REST = Representational State Transfer
- Uses HTTP verbs:

GET	read / search
POST	create
PUT	replace / update
DELETE	delete

- Resources are addressed by URLs:
  - `/Patient/123`
  - `/Observation?code=XYZ&subject=Patient/123`

Simple HTTP Examples

```
GET /fhir/Patient/123 HTTP/1.1
Host: example.org
Accept: application/fhir+json
```

```
POST /fhir/Patient HTTP/1.1
Host: example.org
Content-Type: application/fhir+json

{ ...patient JSON body... }
```

- FHIR defines how servers must behave when these requests are sent

FHIR RESTful Operations

- Instance-level:
  - `GET /Patient/123`
  - `PUT /Patient/123`
  - `DELETE /Patient/123`
- Type-level:
  - `POST /Patient`
  - `GET /Patient?name=smith`
- Whole-system:
  - `GET /metadata`
  - `GET /_history`

Search Parameters

- FHIR defines a set of search parameters per resource type
- Examples:
  - `Patient?name=smith`
  - `Observation?subject=Patient/123`
  - `Observation?code=85354-9`
  - `Observation?date=gt2024-01-01`
- Can be combined:
  - `Observation?code=85354-9&subject=Patient/123`

Error Handling

- HTTP status codes:
  - 200 OK
  - 201 Created
  - 400 Bad Request
  - 404 Not Found
  - 422 Unprocessable Entity
- FHIR may return an `OperationOutcome` resource describing the problem
- In the lab we will intentionally break some JSON to see how the server responds

Lab 1: Basic Interaction

Goals

- Query existing `Patient` and `Observation` resources
- Create a new `Patient`
- Create a new `Observation` for that patient
- Retrieve the created resources

- Implemented via a Jupyter/Colab notebook
- We will work in pairs to support each other

Lab 2: Mini Clinical Workflow

Goals

- Add multiple Observations
- Create a DiagnosticReport referencing those Observations
- Query and verify the entire workflow
- (Optional) Use a Bundle transaction

Short Code Demo (Python/JS)

- Show a minimal script that:
  - Queries a Patient
  - Retrieves Observations
  - Prints a simple report
- Prepare students for Day 2:
  - Web app interacting with FHIR
  - Overview of standards & AI

FHIR & REST References

- HL7 FHIR Official Specification: `https://www.hl7.org/fhir/`
- FHIR RESTful API: `https://www.hl7.org/fhir/http.html`
- FHIR Search: `https://www.hl7.org/fhir/search.html`
- HL7 (organisation): `https://www.hl7.org/`

- Basic REST tutorial (MDN Web Docs): `https://developer.mozilla.org/`
- JSON basics: `https://www.json.org/`

Wrap-Up

- FHIR provides building blocks for interoperable clinical data
- RESTful APIs are the backbone of modern health IT integration
- Today: hands-on experience with basic operations and a mini workflow
- Tomorrow: web app, AI/standards, and your own use case design